

# A Comparative Study of Fruit Detection and Counting Methods for Yield Mapping in Apple Orchards

---

**Nicolai Häni**

Department of Computer Science & Engineering  
University of Minnesota  
Minneapolis, MN 55455  
haeni001@umn.edu

**Pravakar Roy**

Department of Computer Science & Engineering  
University of Minnesota  
Minneapolis, MN 55455  
royxx268@umn.edu

**Volkan Isler**

Department of Computer Science & Engineering  
University of Minnesota  
Minneapolis, MN 55455  
isler@umn.edu

## Abstract

We present a modular end-to-end system for yield estimation in apple orchards. Our goal is to identify fruit detection and counting methods with the best performance for this task. We propose a novel semantic segmentation based approach for fruit detection and counting and perform extensive comparative analysis against other state-of-the-art techniques. This is the first work comparing multiple fruit detection and counting methods head-to-head on the same datasets. Fruit detection results indicate that the semi-supervised method, based on Gaussian Mixture Models, outperforms the deep learning based methods in the majority of the datasets. For fruit counting though, the deep learning based approach performs better for all of the datasets. Combining these two methods, we achieve yield estimation accuracies ranging from 95.56% – 97.83%.

## 1 Introduction

Precision agriculture techniques have ushered in an unprecedented era of automation in farming and food production. There exist now commercial solutions for automating numerous farming tasks such as monitoring crop fields ([Gamaya, 2019]), seeding ([Tanimura and Antle, 2019]), harvesting ([CNH, 2019]), and packaging. However, the application domain of precision agriculture has been primarily limited to commodity crops such as wheat, rice, maize, etc. Developing solutions for specialty crops (such as fruits or vegetables) has been challenging due to the complex geometry of orchards compared to row crops. As it is hard to generalize automation techniques across multiple specialty crops, researchers have focused on crop specific systems. As it is hard to generalize automation techniques across crops, researchers have focused on developing systems for specific crops. For apples, researchers have addressed problems such as diameter estimation [Wang et al., 2013], automated pruning [He and Schupp, 2018] or apple picking [Baeten et al., 2008]. One of the precursors to these tasks is the accurate detection and localization of fruits on trees. In his work, we focus on establishing fruit detection, localization, counting and tracking methods for yield mapping.

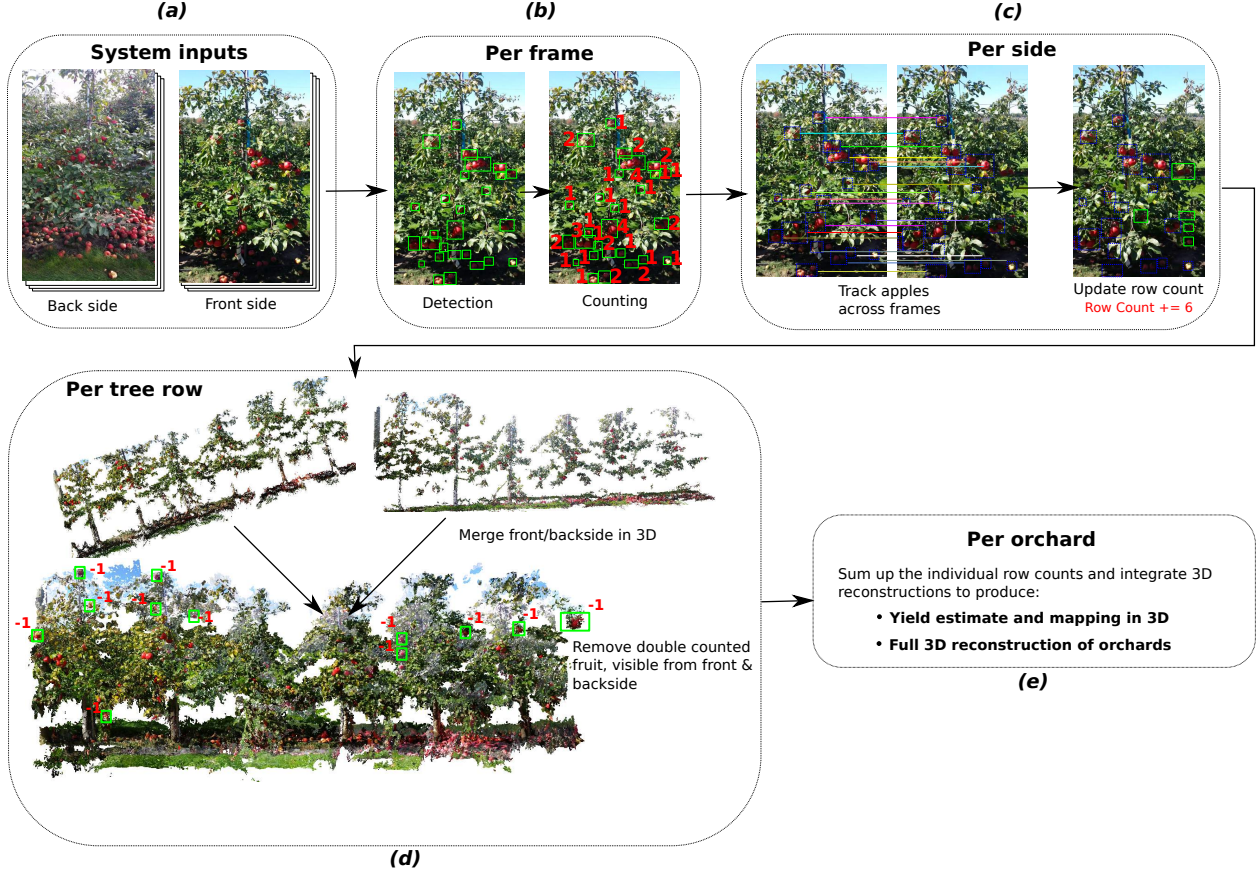


Figure 1: Overview of the yield estimation process. (a) Given two image sequences from the same portion of an orchard row, one from the front and one from the back. (b) Fruits are detected and counted in each frame. (c) Fruits are tracked across the image sequence to avoid double counting. As outputs, we have an image sequence per side, together with fruit locations and counts. (d) We reconstruct each image sequence in 3D and merge the two reconstructions into a single 3D model of the tree row. For yield estimation, we can now remove fruits visible from both sides of the tree row. (e) The pipeline produces yield estimates together with a 3D reconstruction and fruit mapping.

Automated yield mapping is a precursor to numerous precision farming and phenotyping tasks. In the absence of an automated system, growers have to resort to manual data collection. Workers have to sample a few trees at random and extrapolate the counts to the whole orchard, which can lead to erroneous yield estimates. Consequently, significant research has been dedicated to automating the yield mapping process [Bargoti and Underwood, 2017b, Wang et al., 2013, Gongal et al., 2016, Das et al., 2015, Stein et al., 2016, Roy et al., 2018a, Roy et al., 2018b]. In general, these yield mapping systems collect images from a single/both sides of a tree row, detect and count the fruits, track them across multiple frames and merge the fruit counts. Before our work in [Roy et al., 2018a], none merged fruit counts from both sides of the tree row without the help of external navigational sensors. In contrast, we presented a method using semantic information to build a consistent model of the tree row and used the merged reconstruction to combine fruit counts from both sides. This method achieved yield estimation accuracies ranging from 91.98% – 94.81%. An overview of this system is given in Figure 1. In this paper, we use the same end-to-end system for yield mapping but improve several key components of the system, resulting in yield estimation accuracies of 95.56% – 97.83%. Additionally, we provide extensive experimental evaluations and conduct a comparative study with other state-of-the-art methods. Our main technical contributions are the following:

**Fruit detection** We demonstrate how a semantic segmentation network (U-net) can be used to identify

apples and provide performance comparisons with other state-of-the-art methods.

**Fruit counting** We present an improved method over [Häni et al., 2018] for estimating accurate fruit counts at the image level.

**Comparative study of state-of-the-art fruit detection and counting methods** New apple detection and counting approaches proposed in the literature are tested on separate, specialized datasets, making head-to-head comparisons of the results impossible. To alleviate this problem, we evaluate the performance of three fruit detection and two counting methods on the same datasets. We hope that in the future our experiments together with the datasets can form a benchmarking platform.

**Yield estimation** Using the system presented in our recent work [Roy et al., 2018a] we provide complete yield results.

In Section 2 we present related work on the topic of yield estimation, fruit detection, counting, and tracking. The problem formulation together with an overview of the end-to-end yield estimation process used for this paper is presented in Section 3. Section 4 presents the technical details of the individual components. Experimental results, together with the strengths and weaknesses of each approach, are shown in Section 6.

## 2 Related Work

Yield estimation for specialty crops is a challenging problem, which has received much attention over the last few years. The developed approaches range from complete processing pipelines to proposals for a single component, such as fruit detection, counting or tracking. We first discuss standalone yield estimation systems. Next, we take a closer look at approaches that address the fruit detection, counting, and tracking parts.

### 2.1 Systems for Yield Estimation

Hand-designed algorithms used thresholding techniques together with color/shape features to detect fruits. Many of these systems relied on controlled environments for stable fruit detection. [Wang et al., 2013] proposed a system that operates at night with artificial flashlights. To control illumination conditions, [Gongal et al., 2016] developed an over-the-tree sensor system. Fruits in these works were detected using static or dynamic thresholding [Otsu, 1979]. The resulting binary masks were identified as fruits using static features such as eccentricity of Circular Hough Transform [Pedersen, 2007]. For tracking, the first method used a navigational sensor and triangulation, while the second used the 3D information of a time-of-flight camera.

Currently, we see a drastic change in algorithm design in the domain of precision agriculture through the adoption of machine learning techniques. [Das et al., 2015] used a Support Vector Machine (SVM) to classify each image pixel into fruit/background. The detected fruits were tracked across frames using optical flow. To compensate for miscounted fruits, they counted fruits manually on 26 trees and fit a linear model to the data, to adjust the estimated counts. [Roy et al., 2018b] used a Gaussian Mixture Model (GMM), both for fruit detection and counting. Using the tracking method in [Roy et al., 2018a], they reported total yield estimates achieving accuracies ranging from 91.98% – 94.81%.

More recently, deep learning has transformed the field of computer vision and precision agriculture. An early adopter of neural networks, [Stein et al., 2016] proposed a multi-sensor framework to count mangoes. The fruits were detected using a Faster R-CNN (FRCNN) [Ren et al., 2015] network. The position of the mango fruit was estimated using epipolar geometry and GPS information. Additionally, each fruit was assigned to a single tree, using LIDAR generated tree masks. [Bargoti and Underwood, 2017b] used Multilayer Perceptrons and a Convolutional Neural Network (CNN) to segment images into apple/background pixels. The segmented

fruits were counted using a combination of Circular Hough Transform and Watershed Transform [Beucher, 1992]. They sampled images manually at roughly 0.5m to minimize image overlap and avoid double counting. The total yield was estimated by summing up the counts of the individual images. [Liu et al., 2018] used a Fully Convolutional Network (FCN) [Long et al., 2015] to segment images into fruit/background pixels. The individual fruits were tracked across frames using a Kanade-Lucas-Tomasi (KLT) tracker [Lucas and Kanade, 1981] together with the Hungarian algorithm [Kuhn, 1955]. A Structure from Motion (SfM) algorithm was used to compute relative 3D fruit locations and size estimates.

Most of the systems presented so far, partially follow our outline shown in Figure 1 and contain a similar set of components. However, most of them do not build a coherent geometric model from both sides of the row and instead try to find a consistent relationship between single side fruit counts and actual yield. Such systems are bound to overestimate/underestimate fruit counts, specifically in environments where the trees are not well pruned.

## 2.2 Fruit Detection

The first step in a yield mapping pipeline is the accurate detection of fruits. Early methods mostly relied on static color thresholds for detection. The limitations of these methods were often concealed by adding additional sensors, such as thermal- or Near Infrared (NIR) cameras. [Gongal et al., 2015] offers a comprehensive overview of these early detection methods. More recently, [Hung et al., 2015] used a Conditional Random Field (CRF) to segment the image into 5 classes (fruits, trunk, leaves, ground, and sky). Their model achieved F1 detection scores of 0.81. In 2015, Faster R-CNN [Ren et al., 2015] became the de-facto state-of-the-art object detection method. Two recent papers used this network for fruit and vegetable detection. [Bargoti and Underwood, 2017a] used an FRCNN to detect and count almonds, mangoes, and apples, achieving  $F_1$ -scores of  $> 0.9$ . [Sa et al., 2016] used a similar approach to detect green peppers. They merged RGB and Near-Infrared (NIR) data and achieved  $F_1$ -scores of 0.84.

In contrast to these methods, we treat fruit detection as a pixel-wise classification problem and train a semantic segmentation network (U-net) for solving it. We perform a rigorous performance comparison with two state-of-the-art fruit detection methods - a classical semi-supervised approach using GMM [Roy et al., 2018b] and an object detection approach based on Faster R-CNN [Bargoti and Underwood, 2017a].

## 2.3 Fruit Counting

We have mentioned many different approaches for fruit counting already. These approaches often used classical (Circular Hough Transform (CHT)) and object-detection based counting. Using state-of-the-art object detection networks does not require additional counting algorithms. Instead, the network outputs are summed up to receive counts [Bargoti and Underwood, 2017a, Sa et al., 2016, Stein et al., 2016]. Both of these popular counting methods have drawbacks. The main drawbacks of CHT are its reliance on accurate segmentation of the image, inability to handle occlusions and the need to fine tune parameters across datasets. Object detection networks use Non-Maximum Suppression to reject overlapping instances, and often true positives are filtered out. [Bargoti and Underwood, 2017a] found that up to 4% of their error was due to this filtering process.

Alternatively, [Chen et al., 2017], used a combination of two networks for fruit counting. First, an FCN creates feature maps of possible targets in the image. Second, a CNN counts these targets using a regression head. [Maryam Rahnemounfar and Clay Sheppard, 2017] used a CNN for direct counting of red fruits from simulated data. They used synthetic data to train a network on 728 classes to count red tomatoes. Different to these approaches, [Roy and Isler, 2017b] modeled fruit clusters using Gaussian mixture models and presented a method for fruit counting by finding the optimal number of components in the model. This approach is entirely unsupervised, demonstrated qualitative results and achieved an overall counting accuracy of 94.4%. [Roy and Isler, 2017a] presented a method for counting fruits using a robotic manipulator

for active perception.

In contrast to these methods, we formulate fruit counting as a multi-class classification problem. We train a CNN on image patches that represent apple clusters which are then counted by the network in a classification manner with high accuracy. This method is an improvement of our previous work in [Häni et al., 2018]. In this work, we modified the old network to improve accuracy and added extensive experimental validation.

## 2.4 Fruit Tracking

After the fruits are detected and counted, they need to be tracked across frames to avoid double counting. [Wang et al., 2013] used stereo block matching and GPS information to register the fruit locations globally. Similarly, [Stein et al., 2016] used GPS position together with epipolar geometry to accurately track fruits. [Hung et al., 2015] avoided the tracking problem altogether, by choosing a sampling frequency so that images would not overlap at all. While this approach is easy to implement, our previous work in [Roy et al., 2018b] found that using multiple views greatly increases the robustness of fruit detection and counting. [Das et al., 2015] used a multi-sensor platform and optical flow to avoid double counting. In our previous work in [Roy and Isler, 2016], we presented a method for registering apples based on affine tracking [Baker and Matthews, 2004] and incremental Structure from Motion (SfM) [Sinha et al., 2012].

Since fruits are often visible from both sides of a tree row, it is essential to merge the fruit counts from both sides. In our previous work in [Roy et al., 2018a], we addressed this problem. We used semantic information to merge the front and backside of a tree row, and we demonstrated the necessity of this step by analyzing the fruit counts with and without this alignment.

# 3 Problem Formulation and Overview of the Entire System

*“Given two captured image series of the same portion of a tree row from a monocular camera (i.e. cell phone, GoPro etc)- one from the front side of the row and one from the back side- we want to estimate the total fruit counts and locations for the captured portion of the row”.*

An end to end solution to this problem involves solving multiple subproblems such as fruit detection, counting, tracking fruits across multiple views and merging fruit counts from both sides of a row. Many approaches strive to solve this problem - some rely on specialized sensors and hardwares [Das et al., 2015, Gongal et al., 2016], some correlate fruit counts from a single side to ground truth [Stein et al., 2016]. In contrast to these works, we follow the approach proposed by [Roy et al., 2018a, Dong et al., 2018]. Their system detects and counts apples in individual images, and the fruits are tracked across an image sequence using estimated camera motion. Each side of a row is reconstructed independently and merged into a coherent 3D model. With the help of this model, the system eliminates duplicate fruit counts owing to fruits visible from both sides. This is the first approach that can merge fruit counts from both sides of a tree row without relying on any specialized hardware.

## 3.1 Per Frame Fruit Detection and Counting

The per frame fruit detection and counting component takes an individual frame as input and outputs a set of detected fruit clusters and corresponding fruit counts. In this work, we analyze the performance of multiple detection and counting algorithms to find the best-suited method for yield estimation. For detection we propose a novel approach using an image segmentation network (U-Net [Ronneberger et al., 2015]). We compare this network to an object detection network [Bargoti and Underwood, 2017a] and a color-based clustering technique using Gaussian Mixture Models (GMM) [Roy and Isler, 2017b]. We perform an extensive experimental evaluation of all three methods. These results are presented in Section 6.2.

For counting of clustered fruits, we evaluate two approaches. First, we present an improved deep learning based counting approach, for which we presented initial results in [Häni et al., 2018]. Second, we briefly review a classical method of counting by using GMM and image segmentations [Roy and Isler, 2017b]. We perform a comparative analysis of the performance of these methods in Section 6.3.

### 3.2 Tracking Fruits and Merging Fruit Counts Across Multiple Views

Tracking fruits is an essential task to avoid over counting. The tracking component takes the entire sequence of images from a single side of a tree row and the per-frame fruit detections and counts as input. We use the previously proposed method in [Roy et al., 2018a, Dong et al., 2018] for fruit tracking. For completion, we review this component in Section 4.4.

### 3.3 Merging Fruit Counts from Both Sides and Yield Estimation

This component takes the single side reconstructions, and multi-view fruit counts as inputs. It merges the input reconstructions from both sides using semantic information [Roy et al., 2018a, Dong et al., 2018], eliminates double counting owing to fruits visible from both sides of the tree and outputs the total fruit count for the captured portion of the row. Again, for completion, this component is briefly reviewed in Section 4.4.

## 4 Technical Approach

In this section, we provide details for each of the components of the end-to-end yield estimation system discussed in the previous section.

### 4.1 Fruit Detection

At present, deep-learning-based approaches are dominating the field of image segmentation and object detection ([Badrinarayanan et al., 2017, Ronneberger et al., 2015]). They have been effective in fruit and crop segmentation as well ([Chen et al., 2017, Häni et al., 2018]). The overwhelming success of these techniques is often attributed to the huge amount of training data from which the networks learn features that ideally generalize across environments. Obtaining such data in cluttered environments such as orchard settings though is hard and cumbersome. Labeling a  $1920 \times 1080$  image can take up to 15 minutes depending on the number of apples in it. Therefore, it is important to quantify the improvement in detection and counting performance by deep network-based models compared to simpler, classical methods.

Toward this goal, we test three different methods for fruit detection. First, we present a novel approach based on a pixel-wise segmentation network, U-Net [Ronneberger et al., 2015]. We provide details for implementation and rationale for design choices. Second, we revisit an object detection network presented by [Bargoti and Underwood, 2017a], which used a Faster R-CNN [Ren et al., 2015] for object detection. For this, we provide reimplementations with several improvements, such as changing the backbone to a deeper network and the inclusion of a focal loss term [Lin et al., 2017b]. Finally, we review a semi-supervised color-based clustering technique using Gaussian Mixture Models (GMM) and Expectation Maximization (EM) [Moon, 1996] presented in [Roy et al., 2018b]. We perform a thorough side-by-side evaluation of these methods in Section 6.2.

## 4.2 Fruit Detection by Semantic Segmentation

We use a semantic segmentation network to assign a class (apple/background) to each pixel, while minimizing the classification error. The literature contains different networks for dense semantic segmentation [Long et al., 2015, Badrinarayanan et al., 2017]. However, to be effective for fruit detection, a solution must have specific capabilities:

- As data annotation for fruit trees is difficult, the network architecture must be able to leverage small amounts of training data and use the available data efficiently.
- The design must be able to deal with small objects and occlusions. From a typical imaging distance of 2 – 3 meters, apples often occupy 5 – 50 pixels in a  $1920 \times 1080$  image.
- The network must be able to handle class imbalance since the ratio of fruit to background pixels is roughly 1 : 20

A Convolutional Neural Network (CNN) that achieves high precision and recall for small objects with a small amount of data is U-Net [Ronneberger et al., 2015]. U-Net contains a contracting and expanding path, where the contracting path is a typical convolutional network. During the contraction, the spatial information is reduced while feature information is increased. The expansive pathway combines the feature and spatial information through a sequence of up-convolutions and concatenations (skip-connection layers) with high-resolution features from the contracting path. To deal with the problem of class imbalance, we used weighted categorical cross-entropy as our loss function. This loss function allows setting weights that depend on the type of misclassification. For each pair of class labels,  $X$  and  $Y$  one may specify how to penalize label a misclassification of label  $X$  when the actual label is  $Y$ . In our case, the class weights are chosen to be inversely proportional to their frequency in the training data. Additionally, the class weights are normalized. For a schematic overview of our proposed method using this U-Net architecture see Figure 2.

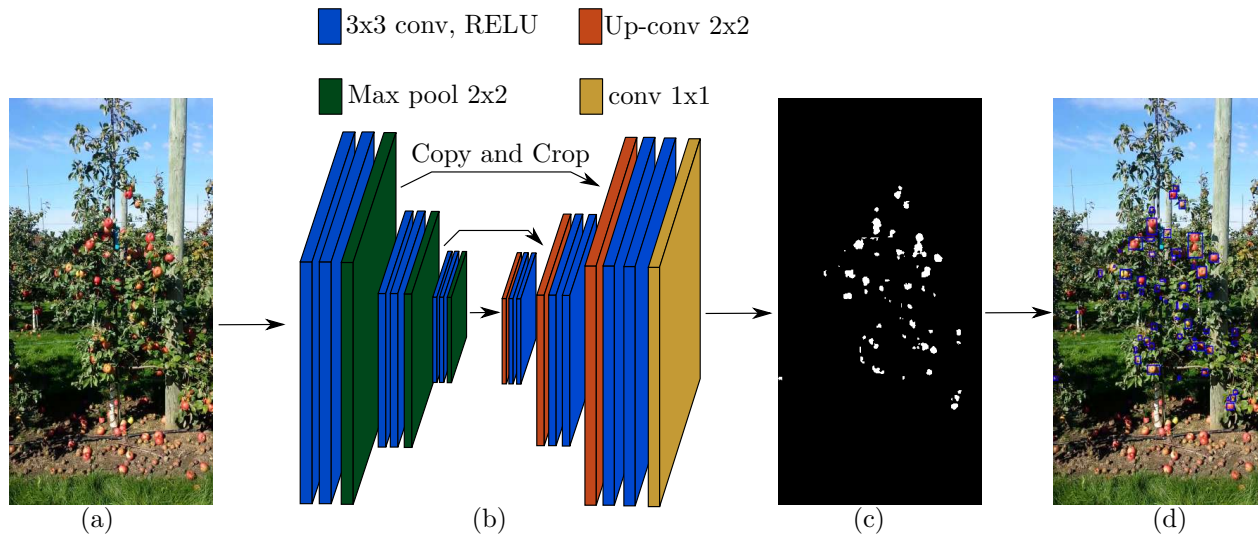


Figure 2: Schematic drawing of U-Net detection workflow. (a) Input image; (b) Schematic view of the down/up-sampling in U-Net [Ronneberger et al., 2015]; (c) The per pixel segmentation of U-Net; (d) Detections after connected component analysis.

**Implementation Details:** All the networks in this work were trained on machines of the Minnesota Supercomputing Institute (MSI) at the University of Minnesota. We used a single node, of which each contains 2 NVIDIA Tesla K20X GPUs, each with 12 GB of memory. The network was implemented in Tensorflow [Abadi et al., 2015], using the Keras [Chollet, 2015] Application Programming Interface (API).

We used a VGG-16 [Simonyan and Zisserman, 2015] network as the backbone in the contracting path. The classification layers were replaced with up-convolutions and concatenation layers for the expansive path. We initialized the contractive path with pre-trained weights of the ImageNet dataset [Russakovsky et al., 2015]. We used images of dimensions  $224 \times 224$  and trained with a batch size of 32 images per GPU for up to 50 epochs. We used Ada-Delta as the optimizer and did not apply any image augmentations.

#### 4.2.1 Fruit Detection by Object Detection Network

Object detection networks simultaneously localize and classify an unknown number of objects in an image. In the literature, two meta-architectures have emerged: Single Shot Multibox Detectors (SSD) [Liu et al., 2016] and two-stage detectors, such as Faster R-CNN [Ren et al., 2015]. Single Shot Detectors use a single feed-forward network to predict object class scores together with bounding box proposals. Two-stage detectors consist of a Region Proposal Network (RPN) and additional network heads. The RPN proposes Regions of Interest (RoI). The second stage computes a classification score together with class-specific bounding box regression for each of these regions. While SSD based models are faster on average than two-stage models, this comes at the cost of detection accuracy [Huang et al., 2017]. This effect is even more pronounced when the objects in question are small. For this reason, mainly two-stage networks have been adopted for the task of fruit detection ([Sa et al., 2016, Stein et al., 2016, Bargoti and Underwood, 2017a]).

For this study, we reimplemented the network proposed by [Bargoti and Underwood, 2017a]. We did not consider [Sa et al., 2016] since they used a combination of RGB and NIR data for prediction. FRCNN has been shown to perform poorly when detecting objects smaller than  $32 \times 32$  pixels. To counteract this problem, we added a Feature Pyramid Network (FPN) with lateral connections [Lin et al., 2017a] to the detection stage of the network. Another modification that we included in the network was a focal loss term, as proposed by [Lin et al., 2017b]. This focal loss is added to the standard cross-entropy term and reduces the loss for easily classifiable examples. For a schematic overview of the network see Figure 3.

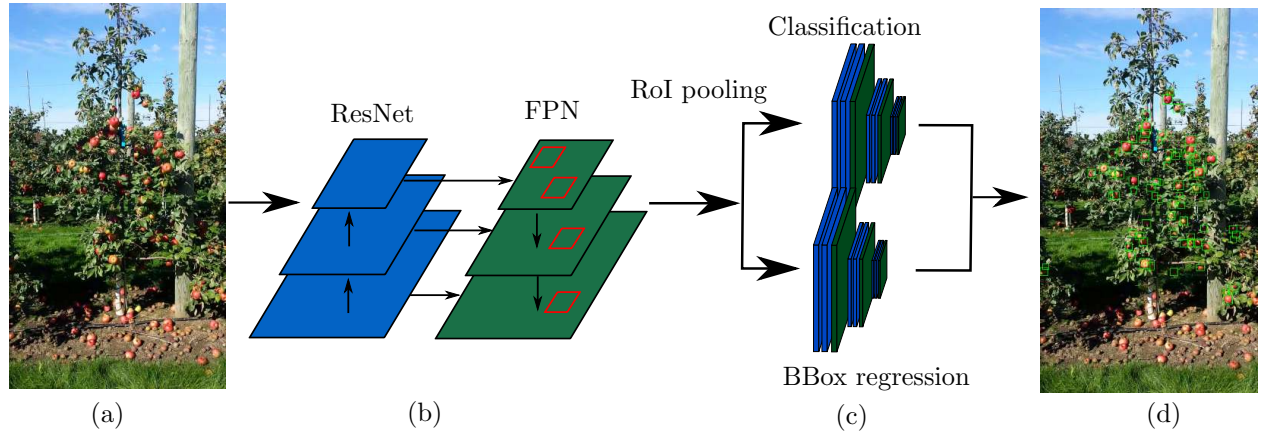


Figure 3: Schematic drawing of Faster R-CNN with FPN head and focal loss. (a) Input image; (b) Region proposal network, using ResNet50 as backbone [Lin et al., 2017a]; (c) Network heads for classification and regression on object detection location; (d) Detections.

**Implementation Details:** The network was trained on a single node, which contains two NVIDIA Tesla K20X GPUs. We used a Tensorflow implementation of Faster R-CNN, which can be found online<sup>1</sup>. We followed the parameters used in [Bargoti and Underwood, 2017a, Ren et al., 2015]. The anchor scales were modified to:  $\{32, 64, 128, 256, 512\}$ . [Bargoti and Underwood, 2017a] used VGG [Simonyan and Zisserman, 2015] as backbone architecture. In this work, we use a ResNet50 [He et al., 2016]. We trained the network for a maximum of 100 epochs, each with 10000 iterations. Images were resized so that the larger size is 500 pixels. We only used left/right flipping as data augmentations.

<sup>1</sup><https://github.com/fizyr/keras-retinanet>

#### 4.2.2 Fruit Detection by Semi-supervised Clustering

It is important to quantify the improvement in detection and counting performance by deep network-based models compared to simpler, classical methods. For this purpose, we previously described methods against [Roy et al., 2018b], who presented a simple method for apple detection, based on semi-supervised clustering with Gaussian Mixture Models (GMM). Here, we briefly revisit the method for completion.

The method over-segments the input image into SLIC super-pixels [Achanta et al., 2010], using the LAB colorspace. Each super-pixel is represented by the mean LAB color of the pixels within the super-pixel. The resulting super-pixels are clustered into approximately 25 color classes. Finally, each super-pixel is classified into apple or background, based on KL divergence [Goldberger et al., 2003] from hand-labeled classes. These classes are obtained by user-supervision. The method provides a user interface, where the user is asked to provide supervision for a few frames, by identifying the classes belonging to apples. The user-supervision allows this method to account for different lighting conditions and the color of the particular apple variety. The steps of this method can be found in Figure 4.

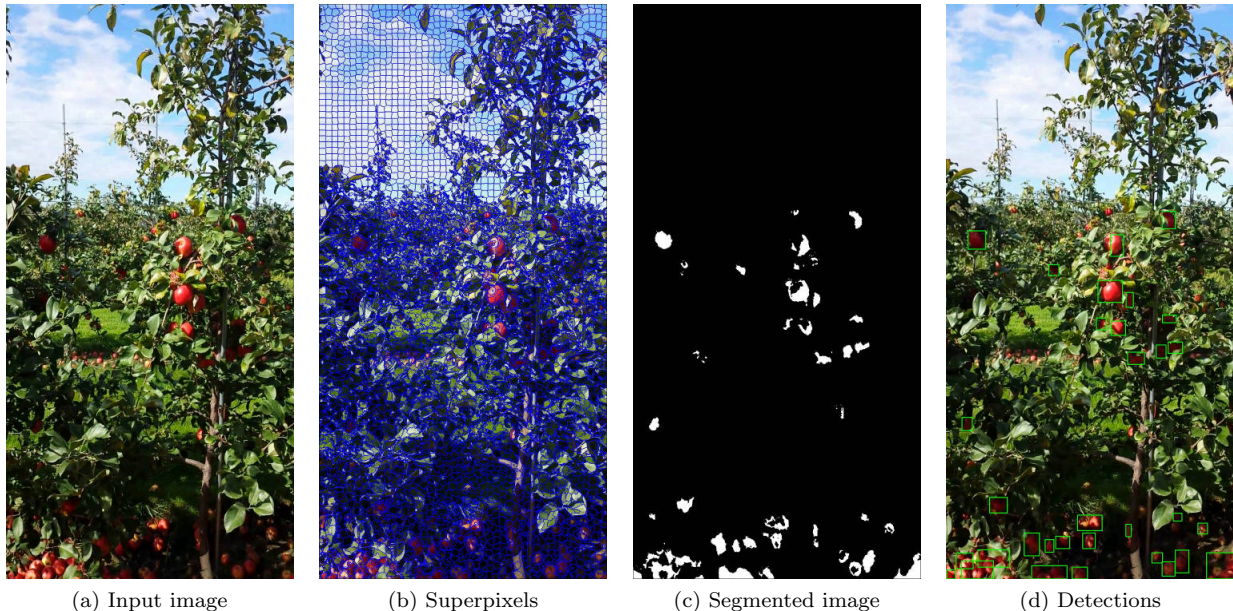


Figure 4: Semi-supervised detection pipeline from [Roy et al., 2018b].

The main advantage of this method is the simplicity of annotation. Typically, 10–20 clicks (user-supervision) are enough to create a classification model to detect all the fruits in a video. However, this method does not work if the fruits are not distinguishable by color from the background. The training model for the GMM is obtained in a user-supervised or semi-supervised fashion. User-supervised means that the first 5 frames of a given dataset were used to train the model. The semi-supervised version of the model was created on a single training dataset, different from the test sets. In section 6.2, we evaluate both of these techniques against the deep learning counterparts.

#### 4.3 Fruit Counting

Once the fruits are detected, we aim to count them in a per frame manner. Challenges that make this task difficult are: (1) Apples often grow in arbitrarily large clusters. (2) Apples are often occluded by branches, leaves or other fruits. As we discussed in Section 2, existing methods for fruit counting are dominated by Circular Hough Transformation (CHT) [Pedersen, 2007]. CHT requires extensive parameter tuning and fails

to handle occlusions. These issues led to the development of more sophisticated methods for fruit counting. In this section, we discuss two such approaches. First, we present an approach where we formulate the fruit counting problem as a multi-class classification task and solve it using a CNN. A preliminary version of this work was presented in [Häni et al., 2018]. In this version, we change the backbone network to a deeper Resnet50 and add extensive experimental evaluation. Second, we review a classical approach using GMM and Expectation Maximization (EM) [Roy and Isler, 2017b, Roy et al., 2018b]. Both of these approaches work with either the semi-supervised or the U-Net detection methods as inputs. With the Faster R-CNN method, fruit counts are equivalent to summing up the individual detections.

### 4.3.1 Counting using Convolutional Neural Network

We approach the problem of accurately estimating clustered apple counts by making the following observations: (1) Apples are sparsely distributed over the whole image; (2) they are often clustered together; (3) and cluster sizes are unevenly distributed. We define the accurate counting of clustered apples as a classification problem with a finite number of classes representing the apple counts per Region of Interest (RoI). This method takes the detected ROI's, that are likely to contain apples, as inputs. The network can receive input from a variety of detection algorithms. For example, any of the methods described in Section 4.1 would produce acceptable image patches. We constrain the maximum size of the apple clusters to six apples. Previous work [Roy et al., 2018b, Häni et al., 2018] has empirically established a cluster size of six apples as a reasonable upper bound. We, therefore, define 7 classes, representing the apple counts per image patch, including zero.

In [Häni et al., 2018] we presented a preliminary version of this method using an AlexNet [Krizhevsky et al., 2017] architecture. In this work, we used a ResNet50 [He et al., 2016] architecture. Additionally, we tested Google's Inception Resnet v2 [Szegedy et al., 2017]. While this network contains  $\sim 56$  million parameters (compared to the  $\sim 26$  million of a ResNet 50), the network only performed 0.4% better on average over all the validation sets. This improvement does not justify the size increase of 215MB compared to the 99MB of the ResNet50. Additionally, the Inception ResNet v2 took longer to train and was slower during inference. We removed the fully connected and the prediction layers, before adding new ones for retraining. The image input layer was fixed to receive images of size  $224 \times 224$  pixels and batch size was 64. Images were loaded so that the larger size was equal to 224 pixels. We kept the image's aspect ratio and padded the image with zeros if necessary. The weights of the network were initialized with pre-trained ones from ImageNet [Russakovsky et al., 2015]. A schematic overview of the proposed counting approach can be seen in Figure 5.

**Implementation Details:** The counting network was trained on a single MSI node, which contains two NVIDIA Tesla K20X GPUs. The network was implemented in Tensorflow [Abadi et al., 2015], using Keras [Chollet, 2015]. For optimization, we used the Adam optimizer with an initial learning rate of 0.001, beta\_1 of 0.9 and beta\_2 of 0.999. We only used left/right flipping as image augmentations and trained the network for up to 50 epochs.

### 4.3.2 Counting using Gaussian Mixture Model

We compare the neural network counting method to one previously proposed in [Roy and Isler, 2017b]. We describe this method for completion. This method takes the segmented apple cluster images as input and outputs fruit counts and locations. It exploits the idea of the images being generated from a two-dimensional world model (probability distribution). Each apple in the input image is modeled by a Gaussian probability distribution function (pdf), and apple clusters are modeled as a mixture of Gaussians. The fruit counts are obtained from the world model configuration that is most likely to generate the image. If the correct number of apples are known a priori, we can find the most likely world model (GMM) using the EM algorithm [Moon, 1996] in this fashion. This method has the added advantage that it can compute locations of individual fruits in an input image. With the assistance of a depth camera, it has the potential to estimate fruit sizes. On the downside, it requires accurate segmentation of fruits and cannot reject false positives.

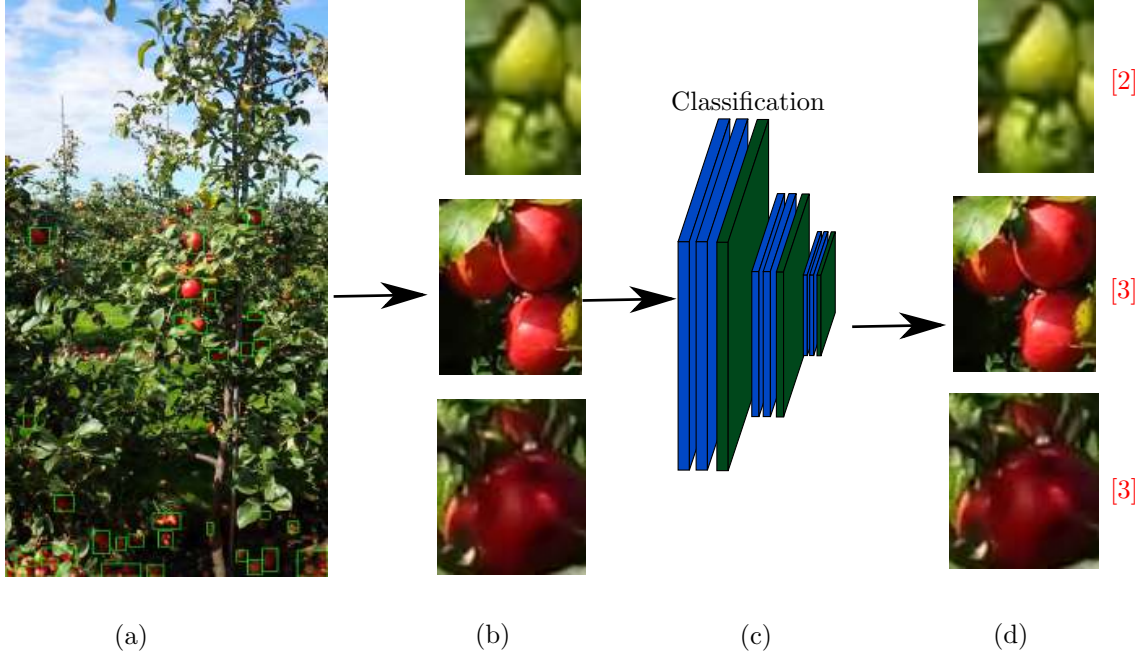


Figure 5: Schematic drawing of counting using classification network. (a) Input image with detected regions of interest; (b) Extracted regions; (c) ResNet50 for classification of image patches; (d) Patches are classified into 7 counts.

#### 4.4 Multi-view Fruit Tracking, Counting and Yield Mapping

After the detection and counting stages, we have per frame fruit counts and their locations. To provide yield estimates, we need to integrate these counts over multiple views throughout the dataset. In addition to tracking the fruits from a single side, we need to register them from both sides of the tree row (some fruits are visible from both sides) to avoid double counting. In cluttered environments, such as apple orchards, the appearance of any given cluster changes drastically between views. It is possible that in some frames, the apples are partially occluded or not visible at all. By tracking clusters across frames and merging the individual predictions, we increase the robustness of our counting approach.

The steps of the tracking algorithm are visualized in Figure 6. These steps were developed as part of an extensive research effort, and we only mention the individual steps in this paper for completeness. To re-implement the multi-view tracking approach, we refer the interested reader to [Roy et al., 2018a, Dong et al., 2018]. Given an input image (Figure 6(a)) we use the detection step to obtain a segmentation mask, containing only the fruits (Figure 6(b)). Afterward, we use Structure from Motion (SfM) to obtain a semi-dense 3D reconstruction of the trees (Figure 6(c)). However, the resulting point cloud does not have a one-to-one correspondence with the pixels in the images. To establish this correspondence, we project the reconstructed point cloud to all camera frames (Figure 6(d)). We compute the intersection of the reprojected image and the binary mask to identify the 3D points belonging to the detected apples (Figure 6(e)). Subsequently, we perform a connected component analysis in 3D (Figure 6(f)). By using these connected components together with the estimated camera poses from SfM, we can reproject the outlines of these connected components onto an image series. These outlines provided us with a series of Region of Interest (RoI) that all show a single cluster of fruits (Figure 6(g)). A 3D cluster may appear in several frames (see Figure 6(g)). We chose the three frames with the highest amount of segmented apple pixels and report the median count of these three frames as the fruit count for the cluster.

We remove the apples on the ground and trees in the background for all the single-side counts by using a computed 3D ground plane and depth masks. We perform these steps for both sides of the row. Figure 7

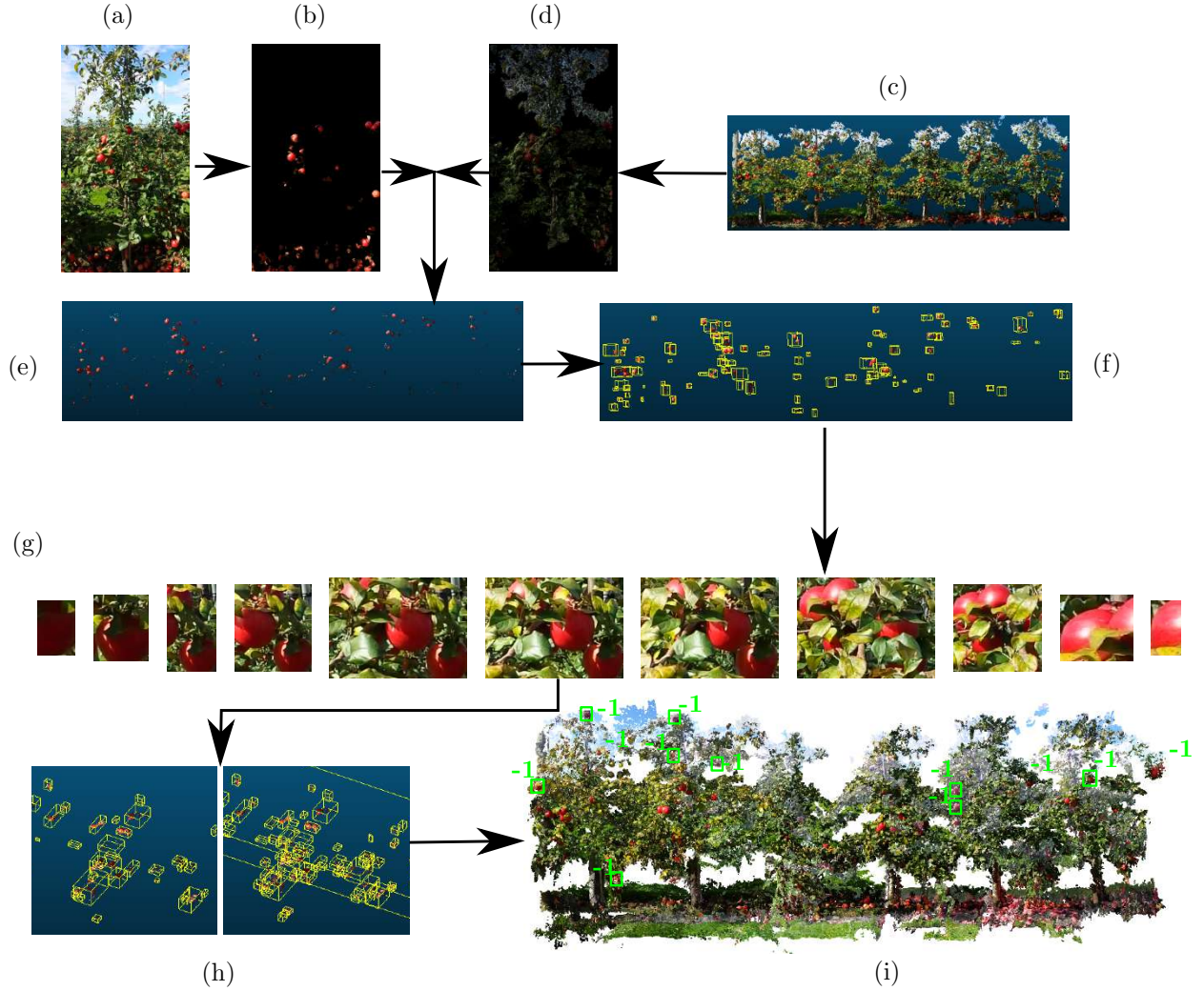


Figure 6: Schematic overview of the fruit tracking and yield estimation pipeline. (a) Input image; (b) Segmentation of the fruits; (c) 3D Reconstruction of a single side of the tree row; (d) Reprojection of fruits on 2D images; (e) Segmented pointcloud; (f) 3D connected components in 3D; (g) Tracked fruit clusters reprojected across multiple frames; (h) Fruit clusters visible from both sides of a tree row; (i) Elimination of double counting for fruits visible from both sides.

shows an overview of this process. Additionally, we need to merge the 3D reconstructions of the front- and back side of the tree row to eliminate double counting of fruits that are visible from both sides. We use the algorithm described in [Roy et al., 2018a, Dong et al., 2018] to accomplish this task. To merge fruit counts from both sides, we compute the intersection of the connected components from both sides (Figure 6(h)). Then, we compute the total counts by summing up the counts from all the connected components, computing the intersection area among them (among 1, 2,..., the total number of intersecting clusters) and adding/subtracting the weighted parts using the Inclusion-Exclusion principle [Andreescu and Feng, 2004] (Figure 6(i)). This method is thoroughly evaluated in Section 6.5.

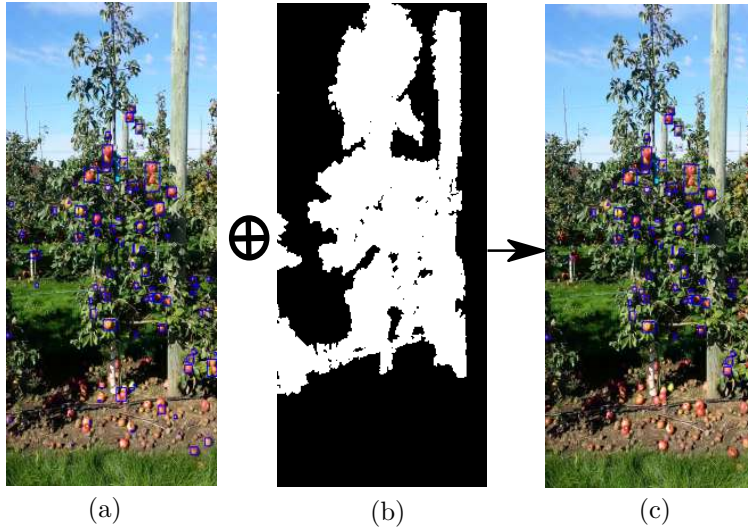


Figure 7: Schematic overview of the removal of fruits on the ground and on trees in the background. (a) All Detections; (b) Mask for removing ground plane and fruits in the background; (c) Filtered detections

## 5 Datasets

All of the data used for this paper were collected at the University of Minnesota Horticultural Research Center (HRC) in Eden Prairie, Minnesota between June 2015 and September 2016. Since this is a university orchard, used for phenotyping research, it is home to a large variety of apple tree species. We collected video footage from different sections of the orchard using a standard Samsung Galaxy S4 cell phone. During data collection, video footage was acquired by facing the camera horizontally at a single side of a tree row. Individual images were extracted from these video sequences. Figure 8 shows the orchard layout and the tagged tree rows. Datasets 1 – 3 use the same tree rows (same apple variety) for training and testing. However, the data was acquired in different years and during different growth stages.

### 5.1 Datasets for Apple Detection / Yield Estimation

**Training Sets:** A total of 10 datasets were sampled from 6 tree rows (see Figure 8a) for training the U-Net/FRCNN detection models. From these 10 datasets we annotated 103 images of size  $1920 \times 1080$  pixels. All of these datasets were acquired in 2015 at the HRC, and they contain different apple varieties, fruits across different growing stages and a variety of tree shapes. See Figure 9(first four from left) for a few sample images from the training sets. To develop a training model for the Gaussian Mixture Model (GMM) based detection without user supervision, we collected an additional dataset in 2015, from the same row of the training set 2. We captured a video from a single side of the row using a Garmin VR camera. Instead of annotating the video manually, we obtained user supervisions in the form of clicks on apples for fifty frames, randomly sampled from the entire video. We refer to this dataset as the "Semi-supervised GMM" dataset

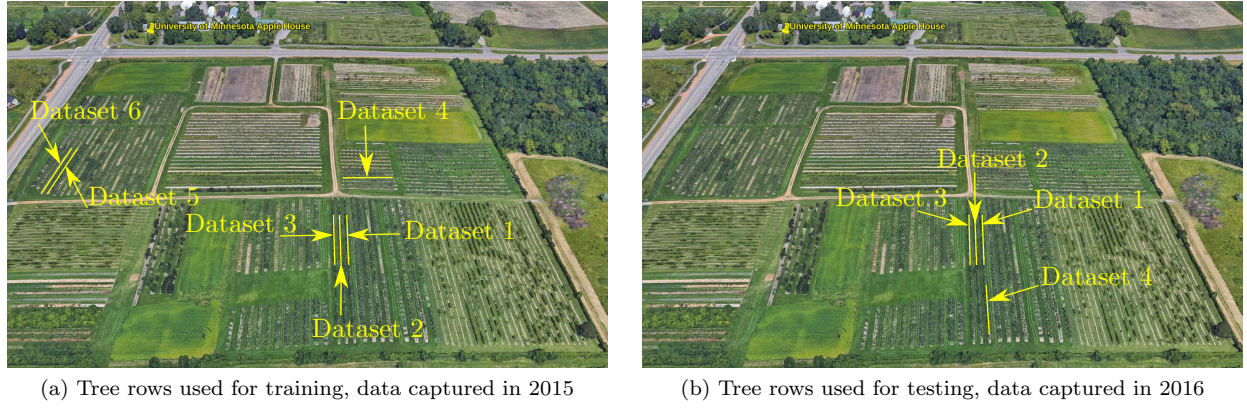


Figure 8: Tree row layouts at the University of Minnesota HRC

for the rest of the paper. See Figure 9(rightmost) for a sample image from this dataset.



Figure 9: Four sample images from the annotated datasets used for training U-Net and FRCNN (first four from left) and a sample image from the Semi-supervised GMM dataset (rightmost).

**Validation Sets:** The validation data were sampled from the same 10 datasets as the training data in the following fashion:

**GMM:** The GMM based method does not require annotated validation data.

**U-Net:** We chose an 80/20 split of the extracted patches for training and validation.

**FRCNN:** We chose an 80/20 split of the extracted patches for training and validation.

**Test Sets:** To evaluate detection and yield estimation performance we arbitrarily chose four different sections of the orchard (see Figure 8b). We collected seven videos from these four segments in 2016. These sections were annotated manually with bounding boxes to mark fruit locations. We also collected ground truth for the rows in question by collecting per tree yield and by measuring fruit diameters after harvest. We used all seven videos for the evaluation of the detection and videos from both sides of datasets 1, 2 and 3 for the yield estimation experiments. See Table 1 for dataset details and see example images in Figure 10.

Table 1: Overview of the apple detection/yield estimation test datasets

| Dataset | Number of trees | Number of harvested apples | Characteristics                                                                                                     |
|---------|-----------------|----------------------------|---------------------------------------------------------------------------------------------------------------------|
| 1       | 6               | 270                        | Red apples,<br>planar geometry (apples are visible from both sides),<br>acquired late in the season (yellow leaves) |
| 2       | 10              | 274                        | Red apples,<br>non planar geometry                                                                                  |
| 3       | 6               | 414                        | Mixture of red and green apples,<br>non planar geometry                                                             |
| 4       | 4               | 568                        | Mixture of red and green apples,<br>non planar geometry<br>Only collected video from the sunny side                 |

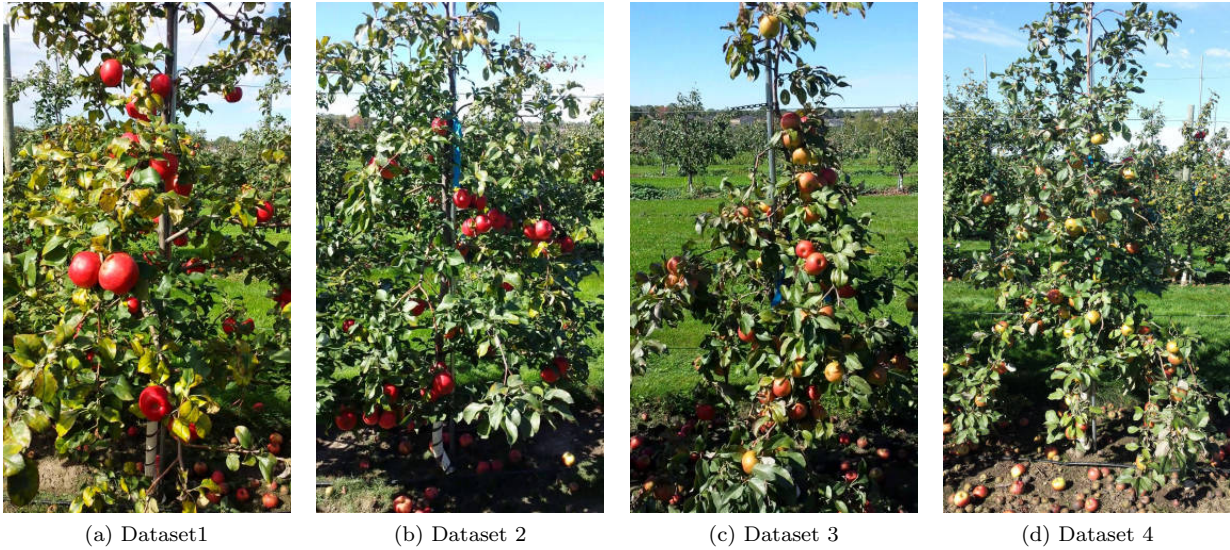


Figure 10: Example images of the datasets used for testing of the detection and yield estimation stages

## 5.2 Datasets for Apple Counting

**Training Sets:** For training of the cluster counting network, we used the same two datasets as in [Häni et al., 2018]. One of these datasets contains green, and one contains red apples. They were acquired from same tree rows as datasets 5 and 6 (see Figure 8a) in 2015. Both datasets were obtained from the sunny side of the tree row. From these two datasets, we extracted image patches using the GMM detection method described in Section 4.2.2. In total, we obtained 13000 image patches, which were annotated manually. Additionally, we extracted 4500 patches at random that do not contain apples. To balance our training dataset between classes we up-sampled the training dataset, using random data augmentation (horizontal flipping, rotations of  $\pm 5^\circ$  and Gaussian smoothing), to a total of  $\sim 65,000$  image patches.

**Validation Sets:** To supervise the network during training, we used a 80/20% split of the available data for training/validation.

**Test Sets:** To validate our patch based counting approach we used four datasets. Compared to the preliminary version presented in [Häni et al., 2018], we modified the data to (1) Subsample the first test dataset. This dataset previously contained 7 times more images than the other three datasets. (2) Remove

patches that show apples lying on the ground. The final datasets are composed of a total of 2874 images. See Table 2 for dataset details.

Table 2: Overview of the apple counting test datasets

| Dataset | Number of image patches | Characteristics                                                                                                                          |
|---------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| 1       | 956                     | Red apples,<br>contains patches from the sunny and shady side of the tree row                                                            |
| 2       | 628                     | Yellow and orange apples,<br>contains patches from the sunny side of the tree row                                                        |
| 3       | 587                     | Green apples,<br>contains patches from the sunny and shady side of the tree row                                                          |
| 4       | 703                     | Red apples,<br>contains patches from the sunny side of the tree row<br>acquired from larger distance (apples appear at lower resolution) |

### 5.3 Manual Annotation of Datasets

To validate our detection and counting methods, we need image level ground truth. The used annotation process differs slightly between training and validation datasets.

**Detection Training Sets for U-Net and FRCNN:** The fruit detection and yield estimation training sets were annotated using the VGG annotator tool [Dutta et al., 2016]. Apples on the foreground trees were annotated using polygons. Apples on the ground and trees in the background were not tagged.

**Detection Training Sets for GMM:** For the semi-supervised and user-supervised GMM the images were over-segmented into 25 color clusters. Then users were provided with an interface where they can click on the fruits. The closest color cluster to the selected pixel color was found, and the rest of the pixels belonging to this color class were shown to the users. Users added color clusters to the model in this fashion until most of the fruits were identified.

**Detection Test Sets:** Fruits in these datasets were annotated using bounding box annotations. For manual annotation, frames were selected arbitrarily every 1 to 3 second from the test videos (frame rate 30 fps), depending on how much the camera moved since the last annotated frame. Apples on the ground and trees in the background were not tagged.

**Counting Training and Test Sets:** Both counting methods for this paper were evaluated on small image patches. The extracted image patches were annotated by hand with a single ground truth count  $\in [0, 6]$ . At least two human labelers annotated each image. Discrepancies were resolved by a third inspection of the image patches in question.

## 6 Experiments and Results

In this section, we evaluate each of the presented methods for fruit detection and counting quantitatively. Additionally, we give some qualitative insights and analyze some common failure cases.

## 6.1 Training Procedure

**Training the GMM:** To evaluate the GMM based detection method with and without user supervision, we developed two models. To evaluate the performance without user supervision, we use the semi-supervised GMM dataset. This dataset was captured on a different year with a different camera. We train the model in a supervised fashion, by adding relevant clusters. The user-supervised model is trained by clicking on apples for the first five frames for each test video.

**Training U-Net:** For training the U-Net, we extracted patches of size  $224 \times 224$  pixels with a stride of 50 pixels from the original images. In total,  $\sim 59000$  annotated patches were extracted for training.

**Training FRCNN:** Our implementation of Faster R-CNN loosely follows [Bargoti and Underwood, 2017a]. Initially, our network was trained on the same data, which is available open source <sup>2</sup>. This data contains a total of 1120 images with  $308 \times 202$  pixels resolution. We used the same train/validation split like the one used in their paper. To offer a fair comparison to our proposed method, we added our own annotated data to the training set. We extracted image patches of size  $500 \times 500$  pixels by moving a sliding window with stride 50 pixels over the annotated images. In total,  $\sim 9000$  annotated patches were extracted for training.

## 6.2 Detection Results

We use three metrics for evaluation purposes: precision, recall and  $F_1$ -measure. These metrics are obtained using true positives (TP), false positives (FP) and false negatives (FN) rates. Recall is a measure of how many relevant objects are selected out of the total number of objects. Formally,  $\text{Recall} = \frac{TP}{TP + FN}$ .

Precision is a measure of how many of the detected objects are relevant. Formally,  $\text{Precision} = \frac{TP}{TP + FP}$ .

The  $F_1$ -measure is the harmonic mean between precision and recall. Formally,  $F1 = 2 \left( \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \right)$ .

We computed these three measures for all seven test datasets over the entire Intersection over Union (IoU) range  $([0.01, 0.99])$ . IoU is defined as the area of overlap between detection and an object instance, divided by the area of the union of the two. The metrics are computed per frame, and we report the average per dataset. Figure 11 shows the recall of all the four detection methods on the seven datasets.

The user-supervised GMM method outperforms the other three approaches on 6 out of seven datasets and is competitive on the last one. This outcome is expected, as the training models were tuned for each test set. The reason it falls behind in Dataset4 (front) is that color features alone were not enough to detect all the fruits. The performance of the semi-supervised GMM depends on how closely the test set color space resembles the training model. In the case of Dataset1 (front) and Dataset4 (front) the color space of the test set was different from the training model and therefore the recall drops substantially. In the other five test sets, the fruit colors were similar to the training set and the model achieves similar recalls to the user-supervised model. Our proposed technique based on U-Net achieves consistently high recall for all the datasets. In Dataset4 (front) it even outperforms the user-supervised GMM. This success can be attributed to the use of non-color features. The FRCNN method also achieves high recall (especially in the low IoU region. This is consistent with [Bargoti and Underwood, 2017a], who suggested to use the FRCNN with  $\text{IoU} = 0.2$ .

When we look at the plots showing the precision in Figure 12 the story is a different one. The user-supervised GMM approach outperforms the other three methods by a large margin in six out of seven test sets. The user-supervised GMM method is the only method to achieve precision values of  $> 90\%$  on all datasets. These high precision can be attributed to conservative user supervisions, which purposefully avoided ambiguous color clusters. The semi-supervised GMM achieves precision values of  $> 90\%$  in five out of seven datasets. For Dataset2 (back) and Dataset4 (front), the precision values fall under  $80\%$  for all IoU levels. Again, this

---

<sup>2</sup><http://data.acfr.usyd.edu.au/ag/treecrops/2016-multifruit/>

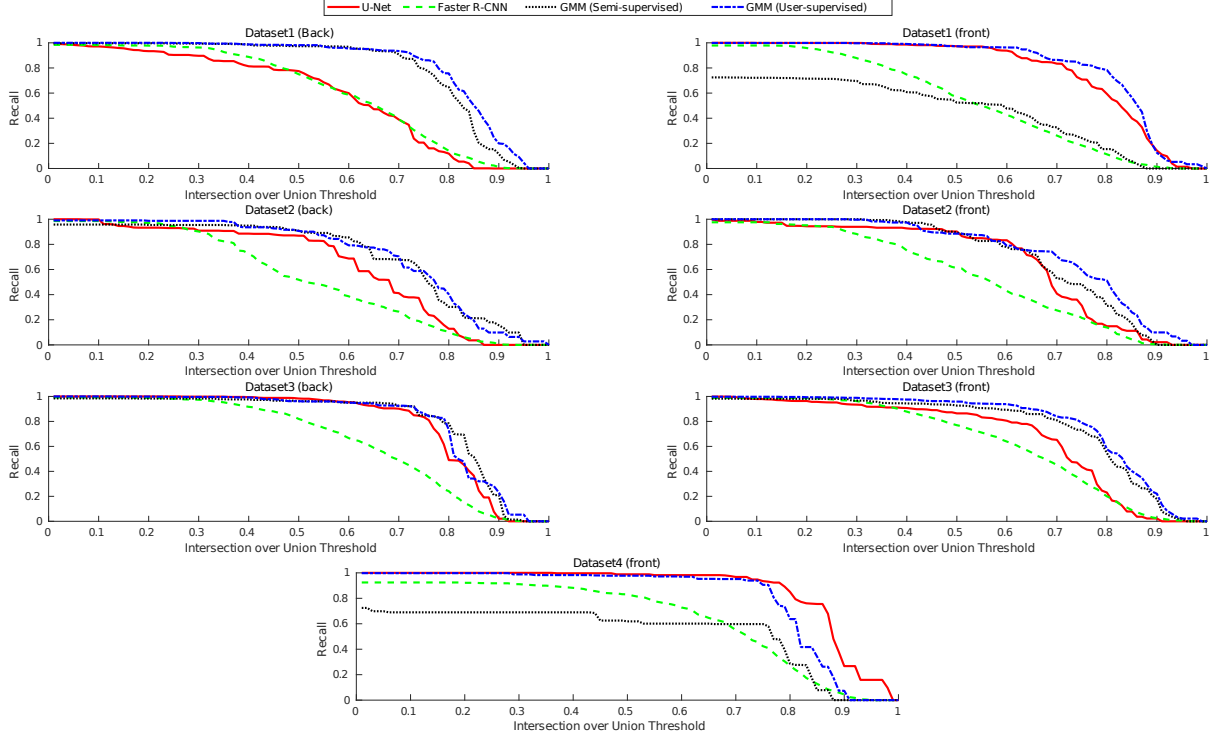


Figure 11: Recall of the detection methods on all seven datasets

drop can be attributed to the dissimilarity in color space and lighting condition between the train and test sets.

The U-Net based approach does not achieve precision values over 80% in four out of seven datasets. We show some examples of kind of false positives in Section 6.4. From the qualitative results, we see that the network detects yellowing leaves as apples. The apple trees did not have any yellowing leaves during 2015, and consequently, our training data did not have any such examples. However, this problem can likely be solved by adding more varied training data. When color features alone were not enough to detect all the fruits, as in Dataset4 (front), the network has higher precision than all other methods. The FRCNN method has the lowest precision on all datasets even with low IoU. This is due to three reasons: First, the network detects more false positives than the other two methods. Second, the FRCNN often merges separate object instances into one. Third, the Non-Maximum Suppression (NMS) either filters out true positives or returns multiple detections for a single object instance. NMS is prone to such behavior due to the manual choice of its threshold. In Section 6.4 we show qualitative examples that illustrate this behavior. These findings again confirm [Bargoti and Underwood, 2017a]. In their work, they estimated that roughly 4% of the total error of their predictions could be attributed to the network’s inability to distinguish clustered fruits. Instead, they treat detections as clusters, not only as single fruits, which helps increase precision.

Since  $F_1$ -measure combines recall and precision, we expect the user-supervised GMM to outperform the other three approaches on most of the datasets as seen in Figure 13. However, the U-Net detection approach achieves competitive  $F_1$ -measure on Dataset3 (back) and even outperforms the GMM approach on Dataset4 (front).

After these extensive evaluations, we can conclude with conviction that when fruits are distinguishable by colors, it is hard to beat the user-supervised GMM model. If color features are not good enough, we do need a more robust solution such as U-Net. However, the training set needs to cover a broader range of apple varieties, growth stages, and lighting conditions.

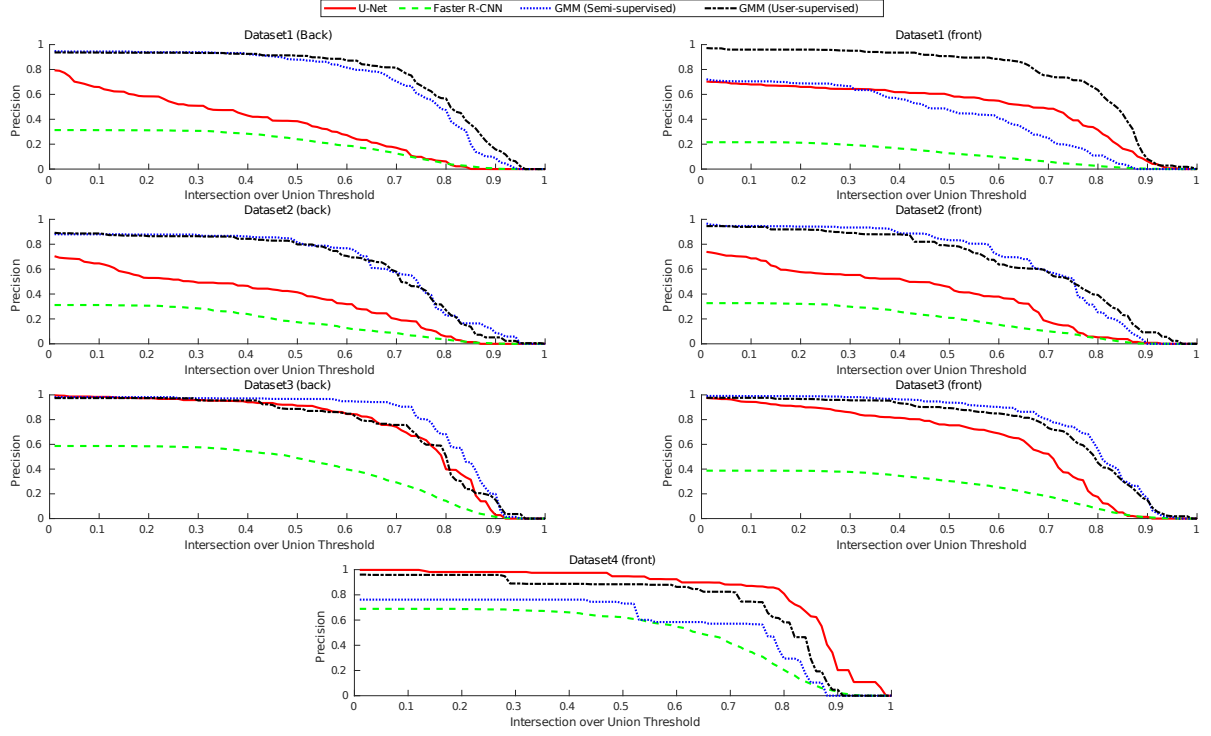


Figure 12: Precision of the detection methods on all seven datasets

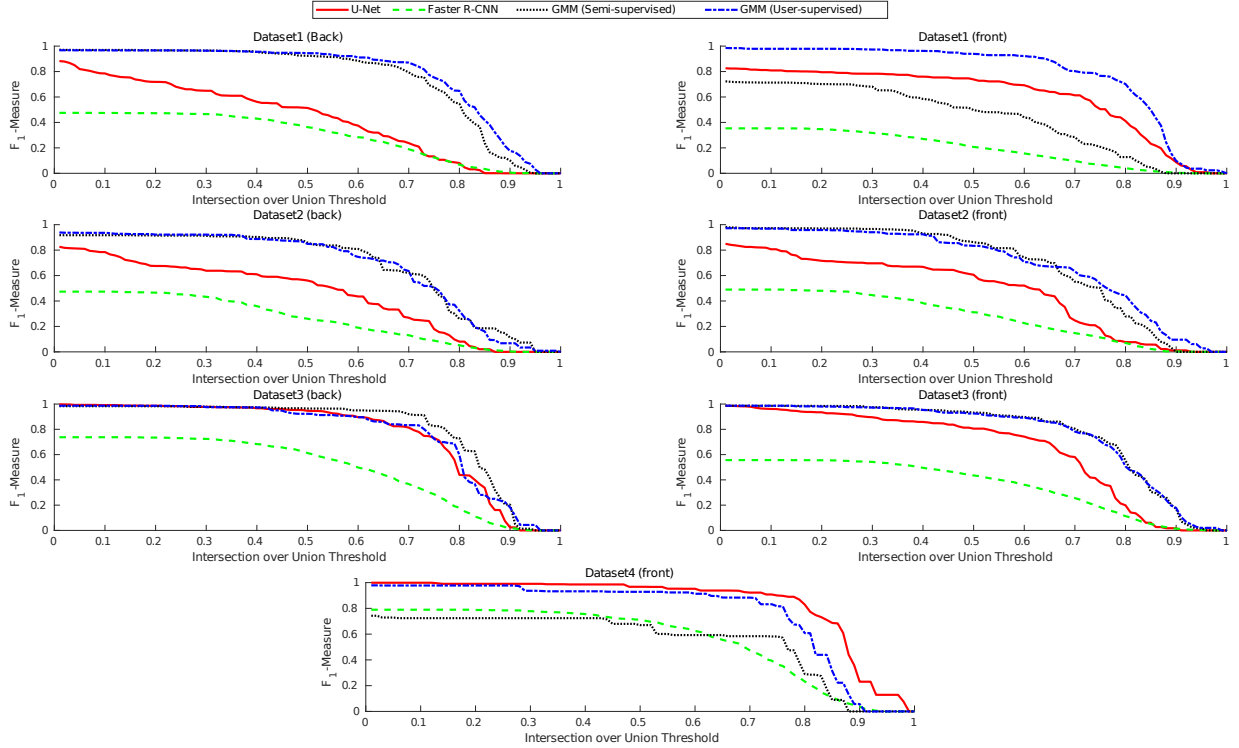


Figure 13: F1-measure of the detection methods on all seven datasets

### 6.2.1 Timing

Detection performance is arguably the most important aspect of an object detection algorithm. A second aspect driving the choice of algorithm is its time complexity. The GMM approach runs at 5 frames per second. The U-Net used in our experiments runs in less than 100 ms per image patch. We use image patches of size  $224 \times 224$  pixels with zero overlap. A full image of size  $1920 \times 1080$  pixels takes less than 4.5 seconds per frame. For the FRCNN we follow the tiling approach proposed by [Bargoti and Underwood, 2017a]. They use image crops of size  $500 \times 500$  pixels with a stride of 50 pixel. The FRCNN network takes 120 ms per image patch. To detect objects on an image of  $1920 \times 1080$  pixels takes up to 46 seconds per frame. We acquire images at a rate of 30 frames per second and move at a speed of 2 m/s which renders the tiling approach infeasible for large orchards. These timings were measured on a conventional laptop with a single NVIDIA Quadro M1000 GPU.

### 6.3 Counting Results

We repeated the experiments of counting apple clusters presented in [Häni et al., 2018] with the modified network. The test datasets were adapted, so the results are not directly comparable. We evaluate counting performance of both methods discussed in Section 4.3. Table 3 shows the counting accuracy.

Table 3: Image Patch Counting Results

| Approach | Test Set 1    | Test Set 2     | Test Set 3    | Test Set 4    |
|----------|---------------|----------------|---------------|---------------|
| GMM      | 88.0 %        | 81.8 %         | 77.2 %        | 76.1 %        |
| ResNet50 | <b>88.8 %</b> | <b>92.68 %</b> | <b>95.1 %</b> | <b>88.5 %</b> |

The ResNet50 network outperforms the GMM model on all of the test sets. On the test set 3 the CNN outperforms the GMM by almost 11%. On test sets 2 and 4 it outperforms the GMM by 18% and 12% respectively. These results show that the proposed neural network generalizes between datasets even under varying illumination conditions and among data sets with different colors. The confusion matrices in Figure 14 show that the GMM is precise in predicting a single apple. For all other categories, the performance of the method drops considerably. In contrast, the neural network’s performance drops slowly in cases of higher fruit counts. Additionally, we see that the deep network can reject false positive detections in 87% of the cases. Comparably, the GMM does so in only 43% of the cases. The distribution of apples in these test datasets is highly skewed towards one single apple. We observed this phenomenon throughout our experiments, as the majority of clusters returned by the detection method are in the range  $[0, 4]$ . This finding suggests that our assumption of seven classes (fruit counts from 0 to 6) was too broad.

### 6.4 Qualitative Results

In this section, we show some qualitative examples from three datasets. We illustrate the performance of the three detection approaches on a sample image from each of these datasets. In Dataset4 (front) color features alone were not enough to detect all the apples; causing problems for the user-supervised GMM. Dataset1 (front) contains many yellowing leaves causing problems for both the U-Net and FRCNN. On Dataset3 (back) both the GMM and U-Net achieved high precision and recall, but the FRCNN still had poor precision.

### 6.5 Yield Estimation Results

Detecting fruits and counting them in a per frame manner are both technically challenging tasks. However, these are only subproblems to the yield estimation task, which we ultimately want to solve. Our goal in this section is to find the best possible system using the previously discussed methods.

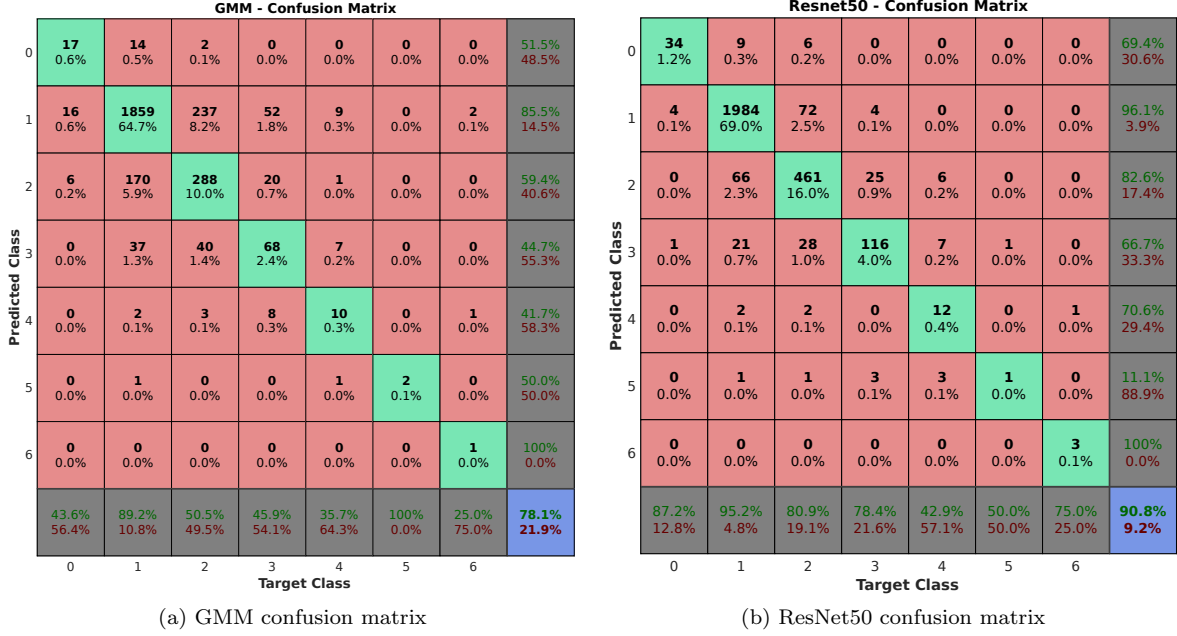


Figure 14: Confusion matrices over all four test datasets

We conduct these experiments solely with the GMM detection method since the U-Net and FRCNN approach did not show satisfactory detection rates. We evaluate which of the discussed counting methods (GMM/ResNet50) is more useful for yield estimation and quantify the overall yield estimation accuracy of the entire system. Additionally, we demonstrate that tracking fruits visible from both sides leads to more consistent results. We use the three test datasets (Dataset 1, Dataset 2, and Dataset 3); where videos from both sides of a row were collected. The computed yield estimates are compared to the harvested ground truth counts. Figure 16b and Table 4 show the shortcomings of adding fruit counts from individual sides independently. These yield estimates vary considerably across datasets for both, the GMM(101.93% ~ 150%) and ResNet50 (103.86% ~ 147.81%). However, this method of summing up counts from both sides achieves the lowest error rate (1.93%) on Dataset 3. Although it does well on this one dataset, it over-counts by up to 50% on others, which makes finding a consistent mapping of these counts to the actual yield tedious and error-prone.

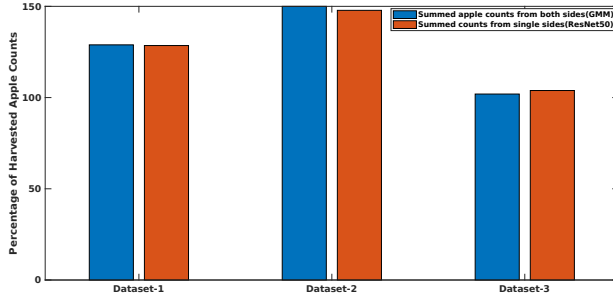
Table 4: Summary of yield results in terms of fruit counts (FCs).

| Datasets  | Harvested FCs | Merged FCs from both sides |                     | Sum of FCs from single sides |               |
|-----------|---------------|----------------------------|---------------------|------------------------------|---------------|
|           |               | GMM                        | ResNet50            | GMM                          | ResNet50      |
| Dataset-1 | 270           | 256 (94.81%)               | <b>258 (95.56%)</b> | 348 (128.89%)                | 347 (128.52%) |
| Dataset-2 | 274           | 252 (91.98%)               | <b>268 (97.81%)</b> | 411 (150%)                   | 405 (147.81%) |
| Dataset-3 | 414           | 392 (94.68%)               | 405(97.83%)         | <b>422 (101.93%)</b>         | 430 (103.86%) |

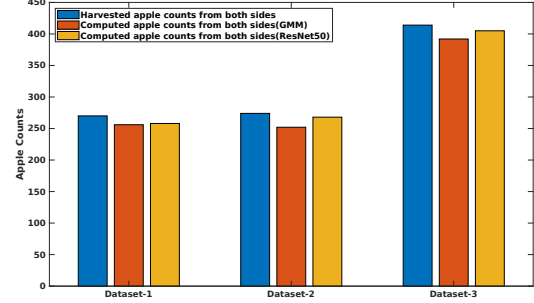
Next, we investigate the performance of the GMM and ResNet50 based counting methods when a consistent geometric representation of both sides of a row is available. As shown in Figure 16 and Table 4, both the GMM (91.98% ~ 94.81%) and ResNet50 (95.56% ~ 97.83%) based counting methods provide more consistent estimates compared to the independently summed fruit counts. ResNet50 achieves better performance than the GMM for all of the datasets, with errors of 2.17% – 4.44% compared to harvested ground truth. These results are even more impressive if we consider that our system has counted only visible apples. The camera does not see any apples that are contained within the tree foliage, and therefore does not count them.



Figure 15: Some qualitative results for the proposed detection methods



(a) Yield mapping by summing up fruit counts from individual sides



(b) Merged fruit counts from both sides

Figure 16: Fruit yield estimation. (a) Independently summed fruit counts from individual sides lead to inconsistent estimates. (b) Merging the fruit counts from both side, we obtain a more consistent estimate from both GMM (91.98%  $\sim$  94.81%) and ResNet50 (95.56%  $\sim$  97.83%) based counting methods.

## 6.6 Failure Cases

In Section 6.4 we presented some qualitative examples. We now analyze the most common failure cases in more detail and offer insights into how these can be overcome in the future.

### 6.6.1 Detection

Some common failure cases of the detection stage are shown in Figure 17.

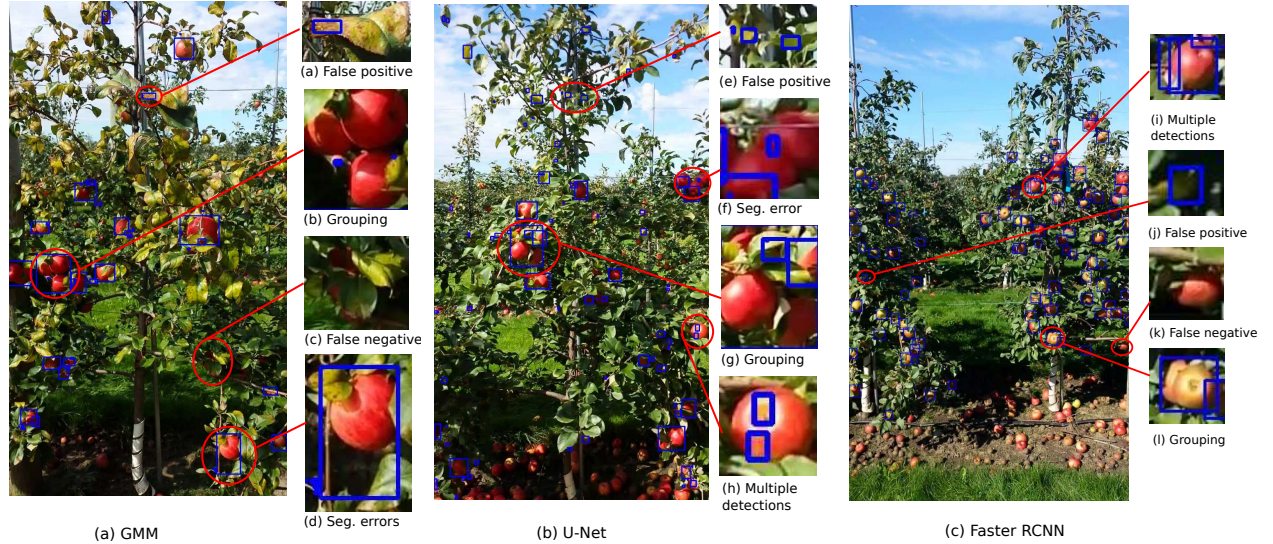


Figure 17: Common failure cases of the three evaluated detection methods

The three methods have similar causes of errors, namely grouping of object instances, false positive detections and false negatives. In addition to these cases, deep learning based methods also split single objects into multiple detections. For the U-Net and the GMM detection methods, the additional network in the counting stage provides the means to reject false positives in  $\sim 85\%$  of the cases. The FRCNN method does not contain an additional network for counting. However, the FRCNN could be changed to classifying instances into cluster counts instead of fruit/background. Such an approach will solve the problem of grouped instances

and can reject false positives. However, doing so is not straightforward. Future research would have to determine how to merge overlapping predictions and set up an adequate training procedure.

The problem of false negatives is a more challenging one. It occurs in all three detection methods, but for different reasons. In the GMM-based detection method, the number of clusters to over-segment the images are chosen by the user a priori. If this threshold is too low, the model lacks the representative power to disambiguate among different object categories. If the threshold is too high, developing a training model become tedious, as many color clusters are required to capture all the fruits. In some cases, the fruits might not be distinguishable by color at all. In the case of the U-Net and the FRCNN, this phenomenon is in part due to a lack of training data. The number of false positives for both of these methods highest on Dataset 1. Acquired in late September, the leaves in this dataset were turning yellow. The change of color impacts the networks' performance due to a lack of similar examples in the training set. An additional reason for false negatives in the FRCNN approach is the usage of Non-Maximum Suppression (NMS). Since NMS uses static thresholds, the network is prone to filter out overlapping true positives. While NMS is the de-facto standard algorithm to reject overlapping instances, it hurts performance when we try to detect individual instances of grouped objects.

### 6.6.2 Counting

The deep learning based counting approach, although achieving overall 90.5% accuracy, contains a few failure cases. Compared to our experiments in [Häni et al., 2018], we removed images where apples were lying on the ground from the test set. Removing such fruits is warranted since we use segmentation mask, obtained from 3D reconstruction, to remove apples on the ground or background trees. Even with these changes and with using a deeper network we cannot remove all failure cases. Figure 18 shows, that errors often happen when fruits are partially visible. This problem can be only be avoided if the detection method returns patches that show only whole fruits. Since fruits are often occluded, this scenario is not realistic. A second problem can be observed in Figure 18b. Here the label was wrongly annotated (it should be 2 instead of three). Additionally, the fruits in this image have substantial overlap. When annotating these images, individual labels are often inconsistent. Human labeling errors are present in most datasets, especially if the annotated scenes are cluttered. To avoid them, we annotate the datasets by multiple people and choose the median annotation. However, this increases the human labeling effort drastically.



Figure 18: Example failure cases of the neural network based counting method

## 7 Conclusion and Future Work

In this paper, we presented new fruit detection and counting methods and evaluate them for the task of yield estimation. One challenge apparent in the literature is that each proposed method uses a separate dataset for testing. To alleviate this problem, we performed a comparative study of different fruit detection and counting methods. This work is the first one to conduct such a comparison on the same datasets. For fruit detection, the semi-supervised clustering technique, based on Gaussian Mixture Model (GMM) achieved the highest  $F_1$ -score in six out of seven datasets. Our segmentation approach based on U-Net performed reasonably well, but the reimplemented Faster R-CNN suffered from poor precision. For fruit counting, the Convolutional Neural Network (CNN) approach was more accurate for both, single image datasets and yield estimation. Additionally, together with our recent work [Roy et al., 2018a, Dong et al., 2018], we presented a complete system for yield estimation. The classical segmentation method combined with the CNN based counting approach achieved yield accuracies ranging from 95.56% – 97.83% compared to the harvested ground truth.

Our results provide quantitative insights into how much we gain in terms of performance with deep learning approaches. For fruit counting, the neural network provides more accurate and robust results. When fruits are distinguishable by color, the evaluated classical detection method outperformed both, the U-Net and FRCNN. The U-Net approach performed exceedingly well when the test dataset was similar to the training dataset (for example U-Net on Dataset-4 (front)). However, for fruit detection, many challenges remain. It has hard to offer conclusive insights towards the generalizability of the U-Net and FRCNN approaches based on the limited amount of data we had available for training. We plan to increase the size of the training data in the future, so that further research can give insights into this question.

Obtaining more data involves labeling fruit boundaries in images. However, labeling by skilled laborers is time intensive and costly. In the future, we plan to explore the use of synthetic data as training data, eliminate the painstaking process of labeling. The advantage of this approach is that labels are readily available. The disadvantage is that models naively trained on synthetic data do not typically generalize to real data.

## Acknowledgements

This work was supported by the USDA NIFA MIN-98-G02. The authors acknowledge the Minnesota Supercomputing Institute (MSI) at the University of Minnesota for providing resources that contributed to the research results reported within this paper <http://www.msi.umn.edu>.

## References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mane, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viegas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., and Zheng, X. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems. page 19.
- Achanta, R., Shaji, A., Smith, K., Lucchi, A., Fua, P., and Ssstrunk, S. (2010). Slic superpixels. Technical report.
- Andreescu, T. and Feng, Z. (2004). Inclusion-Exclusion Principle. In *A Path to Combinatorics for Undergraduates*, pages 117–141. Springer.
- Badrinarayanan, V., Kendall, A., and Cipolla, R. (2017). SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(12):2481–2495.

- Baeten, J., Donn, K., Boedrij, S., Beckers, W., and Claesen, E. (2008). Autonomous Fruit Picking Machine: A Robotic Apple Harvester. In Laugier, C. and Siegwart, R., editors, *Field and Service Robotics*, volume 42, pages 531–539. Springer Berlin Heidelberg, Berlin, Heidelberg.
- Baker, S. and Matthews, I. (2004). Lucas-Kanade 20 Years On: A Unifying Framework. *International Journal of Computer Vision*, 56(3):221–255.
- Bargoti, S. and Underwood, J. (2017a). Deep fruit detection in orchards. In *Robotics and Automation (ICRA), 2017 IEEE International Conference on*, pages 3626–3633. IEEE.
- Bargoti, S. and Underwood, J. P. (2017b). Image segmentation for fruit detection and yield estimation in apple orchards. *Journal of Field Robotics*.
- Beucher, S. (1992). THE WATERSHED TRANSFORMATION APPLIED TO IMAGE SEGMENTATION. *SCANNING MICROSCOPY-SUPPLEMENT*, page 26.
- Chen, S. W., Shivakumar, S. S., Dcunha, S., Das, J., Okon, E., Qu, C., Taylor, C. J., and Kumar, V. (2017). Counting Apples and Oranges With Deep Learning: A Data-Driven Approach. *IEEE Robotics and Automation Letters*, 2(2):781–788.
- Chollet, F. (2015). *Keras*.
- CNH (2019). *CNH Industrial Autonomous Tractor*.
- Das, J., Cross, G., Qu, C., Makineni, A., Tokekar, P., Mulgaonkar, Y., and Kumar, V. (2015). Devices, systems, and methods for automated monitoring enabling precision agriculture. pages 462–469. IEEE.
- Dong, W., Roy, P., and Isler, V. (2018). Semantic Mapping for Orchard Environments by Merging Two-Sides Reconstructions of Tree Rows. *arXiv preprint arXiv:1809.00075*.
- Dutta, A., Gupta, A., and Zissermann, A. (2016). *VGG Image Annotator (VIA)*.
- Gamaya (2019). *Gamaya*.
- Goldberger, J., Gordon, S., Greenspan, H., and others (2003). An Efficient Image Similarity Measure Based on Approximations of KL-Divergence Between Two Gaussian Mixtures. In *ICCV*, volume 3, pages 487–493.
- Gongal, A., Amatya, S., Karkee, M., Zhang, Q., and Lewis, K. (2015). Sensors and systems for fruit detection and localization: A review. *Computers and Electronics in Agriculture*, 116:8–19.
- Gongal, A., Silwal, A., Amatya, S., Karkee, M., Zhang, Q., and Lewis, K. (2016). Apple crop-load estimation with over-the-row machine vision system. *Computers and Electronics in Agriculture*, 120:26–35.
- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, Las Vegas, NV, USA. IEEE.
- He, L. and Schupp, J. (2018). Sensing and Automation in Pruning of Apple Trees: A Review. *Agronomy*, 8(10):211.
- Huang, J., Rathod, V., Sun, C., Zhu, M., Korattikara, A., Fathi, A., Fischer, I., Wojna, Z., Song, Y., Guadarrama, S., and Murphy, K. (2017). Speed/Accuracy Trade-Offs for Modern Convolutional Object Detectors. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3296–3297, Honolulu, HI. IEEE.
- Hung, C., Underwood, J., Nieto, J., and Sukkarieh, S. (2015). A Feature Learning Based Approach for Automated Fruit Yield Estimation. In Mejias, L., Corke, P., and Roberts, J., editors, *Field and Service Robotics*, volume 105, pages 485–498. Springer International Publishing, Cham.

- Häni, N., Roy, P., and Volkan, I. (2018). Apple Counting using Convolutional Neural Networks. In *Intelligent Robots and Systems (IROS), 2018 IEEE International Conference on*. IEEE.
- Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2017). ImageNet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6):84–90.
- Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1-2):83–97.
- Lin, T.-Y., Dollar, P., Girshick, R., He, K., Hariharan, B., and Belongie, S. (2017a). Feature Pyramid Networks for Object Detection. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 936–944, Honolulu, HI. IEEE.
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollar, P. (2017b). Focal Loss for Dense Object Detection. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 2999–3007, Venice. IEEE.
- Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., and Berg, A. C. (2016). SSD: Single Shot MultiBox Detector. In Leibe, B., Matas, J., Sebe, N., and Welling, M., editors, *Computer Vision ECCV 2016*, volume 9905, pages 21–37. Springer International Publishing, Cham.
- Liu, X., Chen, S. W., Aditya, S., Sivakumar, N., Dcunha, S., Qu, C., Taylor, C. J., Das, J., and Kumar, V. (2018). Robust Fruit Counting: Combining Deep Learning, Tracking, and Structure from Motion. *arXiv:1804.00307 [cs]*.
- Long, J., Shelhamer, E., and Darrell, T. (2015). Fully convolutional networks for semantic segmentation. pages 3431–3440. IEEE.
- Lucas, B. D. and Kanade, T. (1981). An Iterative Image Registration Technique with an Application to Stereo Vision. In *IJCAI*.
- Maryam Rahnemoonfar and Clay Sheppard (2017). Deep Count: Fruit Counting Based on Deep Simulated Learning. *Sensors*, 17(12):905.
- Moon, T. K. (1996). The expectation-maximization algorithm. *IEEE Signal processing magazine*, 13(6):47–60.
- Otsu, N. (1979). A Threshold Selection Method from Gray-Level Histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1):62–66.
- Pedersen, S. J. K. (2007). Circular Hough Transform. *Aalborg University, Vision, Graphics, and Interactive Systems*, 123:6.
- Ren, S., He, K., Girshick, R., and Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. In Cortes, C., Lawrence, N. D., Lee, D. D., Sugiyama, M., and Garnett, R., editors, *Advances in Neural Information Processing Systems 28*, pages 91–99. Curran Associates, Inc.
- Ronneberger, O., Fischer, P., and Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer.
- Roy, P., Dong, W., and Isler, V. (2018a). Registering Reconstructions of the Two Sides of Fruit Tree Rows. In *Intelligent Robots and Systems (IROS), 2018 IEEE International Conference on*. IEEE.
- Roy, P. and Isler, V. (2016). Surveying apple orchards with a monocular vision system. In *International Conference on Automation Science and Engineering (CASE)*, pages 916–921. IEEE.
- Roy, P. and Isler, V. (2017a). Active view planning for counting apples in orchards. In *Intelligent Robots and Systems (IROS), 2017 IEEE/RSJ International Conference on*, pages 6027–6032. IEEE.

- Roy, P. and Isler, V. (2017b). Vision-Based Apple Counting and Yield Estimation. In Kuli, D., Nakamura, Y., Khatib, O., and Venture, G., editors, *2016 International Symposium on Experimental Robotics*, volume 1, pages 478–487. Springer International Publishing, Cham.
- Roy, P., Kislay, A., Plonski, P. A., Luby, J., and Isler, V. (2018b). Vision-Based Preharvest Yield Mapping for Apple Orchards. *ArXiv e-prints*.
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., and Fei-Fei, L. (2015). ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision*, 115(3):211–252.
- Sa, I., Ge, Z., Dayoub, F., Upcroft, B., Perez, T., and McCool, C. (2016). DeepFruits: A Fruit Detection System Using Deep Neural Networks. *Sensors*, 16(8):1222.
- Simonyan, K. and Zisserman, A. (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. *ICLR*.
- Sinha, S. N., Steedly, D., and Szeliski, R. (2012). A Multi-stage Linear Approach to Structure from Motion. In Hutchison, D., Kanade, T., Kittler, J., Kleinberg, J. M., Mattern, F., Mitchell, J. C., Naor, M., Nierstrasz, O., Pandu Rangan, C., Steffen, B., Sudan, M., Terzopoulos, D., Tygar, D., Vardi, M. Y., Weikum, G., and Kutulakos, K. N., editors, *Trends and Topics in Computer Vision*, volume 6554, pages 267–281. Springer Berlin Heidelberg, Berlin, Heidelberg.
- Stein, M., Bargoti, S., and Underwood, J. (2016). Image Based Mango Fruit Detection, Localisation and Yield Estimation Using Multiple View Geometry. *Sensors*, 16(11):1915.
- Szegedy, C., Ioffe, S., Vanhoucke, V., and Alemi, A. A. (2017). Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning. *AAAI*, page 7.
- Tanimura and Antle, . (2019). *Plant Tape*.
- Wang, Q., Nuske, S., Bergerman, M., and Singh, S. (2013). Automated Crop Yield Estimation for Apple Orchards. In Desai, J. P., Dudek, G., Khatib, O., and Kumar, V., editors, *Experimental Robotics*, volume 88, pages 745–758. Springer International Publishing, Heidelberg.